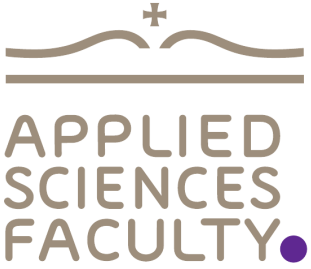
C++ — додаток. Qt. Lecture XX — керування процесами





©Олег Фаренюк,
Creative Commons - Attribution Share Alike (CC BY-SA)

QProcess — абстракція процесів у Qt

- Одна програма може запустити іншу та спілкуватися з нею за допомогою класу QProcess.
- Запустити дочірній процес можна за допомогою методу start() (також можна використати метод open(), який є синонімом, і присутній для сумісності з QIODevice, нащадком якого є QProcess). start() приймає назву програми та її командний рядок. Також їх можна встановити за допомогою setProgram() і setArguments().

  ```
  myProcess->start(program, arguments);
  ```

  Запущений таким чином процес є дочірнім по відношенню до поточного, тому при завершенні поточного буде закритим. Якщо його слід запустити незалежним (приклад – демони Unix-подібних систем), можна використати:

  ```
  myProcess->startDetached(program, arguments);
  ```

- Для startDetached є статичний варіант цього методу, для start існує статичний аналог execute – який очікує на завершення дочірнього процесу та повертає його код завершення.
- QProcess є нащадком QIODevice, тому для обміну даних може використовувати його методи: read(), readLine(), readAll(), getChar(), write(), тощо.

- Слід пам'ятати, що методи QProcess, які читають, читатимуть те, що дочірня програма записувала в свій вивід, а ті, які записують, передають свої дані на вхід дочірній програмі, щоб вона могла їх прочитати. (startDetached "розриває" цей зв'язок!)
- Взаємодія із дочірньою програмою відбувається за допомогою каналів (pipes). По замовчуванню існує два канали виводу (програм) — stdout і stderr.
- Функції читання взаємодіють із поточним каналом, вибрати котрий можна за допомогою методу:

  void QProcess::setReadChannel(ProcessChannel channel);

  який може отримувати `QProcess::StandardOutput` або `QProcess::StandardError`.
- Керувати поведінкою каналів можна за допомогою:

  void QProcess::setProcessChannelMode(ProcessChannelMode mode)

  яка підтримує наступні режими: SeparateChannels – stdout і stderr роз'єднані, MergedChannels – все виводитиметься в stdout, stderr не отримуватиме нічого, ForwardedChannels – **вивід-вивід перенаправлятиметься батьківському процесу**, ForwardedChannels – батьківському процесу перенаправляється лише stderr (Qt 5.2+), аналогічний до нього ForwardedOutputChannel – батьківському процесу перенаправляється лише stdout

- Інші важливі методи керування середовищем дочірнього процесу: setWorkingDirectory та setProcessEnvironment, (для роботи із змінними середовища використовується клас QProcessEnvironment).
- QProcess володіє рядом сигналів та слотів, що дозволяють взаємодіяти із ним.
- Слоти:
  - kill() — завершити процес негайно.
  - terminate() — спроба завершити процес. Під Windows всім головним вікнам процесу приходить повідомлення `WM_CLOSE`, під Unix та Mac OS X — сигнал `SIGTERM`.
- Сигнали:
  - `error(QProcess::ProcessError error)` — генерується у відповідь на помилку дочірнього процесу.
  - finished(int exitCode, QProcess::ExitStatus exitStatus)| — повідомляє про завершення процесу. exitCode – його кода завершення, ExitStatus містить інформацію, Як це завершення відбулося: NormalExit чи CrashExit. exitCode осмислений тільки якщо статус NormalExit.
  - `readyReadStandardError()` і `readyReadStandardOutput()` генеруються, коли є дані для читання із stderr/stdout дочірнього процесу, відповідно.
  - `started()` — генерується, коли процес успішно запущено і його статус Running. (Інші можливі статуси – NotRunning, Starting.)

- start() асинхронна, але іноді зручно мати можливість дочекатися початку роботи процесу. За це відповідає наступна функція:

  bool QProcess::waitForStarted(**int** msecs = 30000)

  Вона блокує виконання поточного потоку, поки QProcess не згенерує сигнал started(), але не більше ніж вказану кількість мілісекунд. Якщо їй передати -1, чекатиме вічно. Повертає true, якщо процес запустився успішно, false в іншому випадку.
- Аналогічна функція для очікування на завершення процесу:

  bool QProcess::waitForFinished(**int** msecs = 30000)

  Повертає true, якщо процес успішно завершився, false в іншому випадку — якщо таймаут, якщо сталася якась помилка, якщо процес вже було завершено.
- waitForReadyRead і waitForBytesWritten — перевизначення відповідних функцій з QIODevice.
- Статус процесу можна перевірити методом state(), а останню помилку, що із ним відбулася, взнати за допомогою: error(). Можливі помилки: FailedToStart, Crashed, Timedout, WriteError, ReadError, UnknownError.

IPC

- http://qt-project.org/doc/qt-5/ipc.html

Посилання

- http://qt-project.org/doc/qt-5/qprocess.html — QProcess.
- http://qt-project.org/doc/qt-5/ipc.html — "Inter-Process Communication in Qt"
- http://qt-project.org/doc/qt-5/qsharedmemory.html — QSharedMemory. Див. також http://en.wikipedia.org/wiki/Shared_memory та приклади: http://developer.nokia.com/community/wiki/QSharedMemory_example, http://qt-project.org/doc/qt-5/qtcore-sharedmemory-example.html
- Пара http://qt-project.org/doc/qt-5/qlocalsocket.html — QLocalSocket та http://qt-project.org/doc/qt-5/qlocalserver.html — QLocalServer.
- Пара http://qt-project.org/doc/qt-5/qtcpserver.html — QTcpServer, та http://qt-project.org/doc/qt-5/qtcpsocket.html — QTcpSocket.
- Пара http://qt-project.org/doc/qt-5/qudpserver.html — QUdpServer, та http://qt-project.org/doc/qt-5/qudpsocket.html — QUdpSocket. http://qt-project.org/doc/qt-5/qabstractsocket.html — QAbstractSocket,
- http://en.wikipedia.org/wiki/Inter-process_communication – Inter-process communication.



